

Sesión 3

Sockets Básicos y Sniffers

Programación para Ciberseguridad — Fase III

Python Ofensivo y Defensivo — Semana 9

Objetivo de la sesión

- Programar un cliente y un servidor TCP con socket
- Entender la diferencia entre TCP y UDP
- Construir un sniffer básico con sockets crudos
- Interpretar un encabezado IP con struct

Cliente TCP

```
import socket

cliente = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
cliente.connect(("example.com", 80))
cliente.send(b"GET / HTTP/1.1\r\nHost: example.com\r\n\r\n")
respuesta = cliente.recv(4096)
print(respuesta.decode())
cliente.close()
```

TCP vs. UDP

SOCK_STREAM	TCP: orientado a conexión, confiable
SOCK_DGRAM	UDP: sin conexión, más rápido, sin garantía de entrega

Servidor TCP

```
import socket

servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
servidor.bind(("0.0.0.0", 9999))
servidor.listen(5)

while True:
    cliente, direccion = servidor.accept()
    print(f"Conexion desde {direccion}")
    cliente.send(b"Hola desde el servidor\n")
    cliente.close()
```

Aplicación en ciberseguridad

Command and Control (C2)

El patrón cliente/servidor es la base de herramientas de C2, usadas tanto en pruebas de penetración autorizadas como en malware real — el código es idéntico; la autorización y el contexto distinguen una prueba legítima de un ataque.

Sockets crudos: un sniffer básico

```
import socket

# Requiere privilegios de administrador/root
s = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_TCP)

while True:
    paquete, direccion = s.recvfrom(65535)
    print(f"Paquete de {direccion}, tamaño: {len(paquete)} bytes")
```

Error común

Un SOCK_RAW sin privilegios elevados falla — da acceso a tráfico que no le pertenece al proceso, por diseño una capacidad sensible del sistema operativo.

Interpretar el encabezado IP

```
import struct
import socket

def decodificar_encabezado_ip(datos):
    encabezado = struct.unpack('!BBHHBBH4s4s', datos[:20])
    ttl = encabezado[5]
    protocolo = encabezado[6]
    origen = socket.inet_ntoa(encabezado[8])
    return {"ttl": ttl, "protocolo": protocolo, "origen": origen}
```

Tip

Interpretar bytes a mano es un buen ejercicio, pero en la práctica Scapy (próxima sesión) hace esto automáticamente y de forma más robusta.

Cierre de la sesión

1. `socket.socket()` con `AF_INET + SOCK_STREAM` crea un socket TCP
2. El patrón cliente/servidor es la base de herramientas de C2
3. `SOCK_RAW` da acceso a paquetes completos, pero requiere privilegios
4. `struct` interpreta bytes crudos según su estructura binaria